ACME Protocol: Decentralized Infrastructure for Bitcoin-Native Art and Asset Indexing

A Technical Whitepaper on Trustless API Routing and Fee Distribution Architecture

Version: 1.0

Date: November 2025

Authors: [ACME Protocol Research Team]

Abstract

The ACME Protocol (Art Coded on the Monetary Engine) presents a novel infrastructure layer for Bitcoin-native digital asset indexing and distributed exchange. This whitepaper describes two critical architectural components: (1) a decentralized DNS routing system employing on-chain commitments and threshold cryptography to ensure clients connect only to verified nodes, and (2) a Bitcoin-native fee distribution mechanism that achieves trustless reward allocation without external tokens or central coordinators. Through the synthesis of Bitcoin's scripting primitives, threshold signature schemes, and periodic checkpointing, ACME achieves Byzantine fault tolerance while maintaining computational efficiency. This work demonstrates how pure Bitcoin script can enforce complex conditional payment logic, enabling a fully decentralized protocol infrastructure that scales with the protocol's transaction volume.

Keywords: decentralized infrastructure, Bitcoin-native design, threshold cryptography, trustless reward distribution, Byzantine fault tolerance, sidechain indexing

---

1. Introduction

1.1 Motivation and Context

The proliferation of off-chain protocols and sidechains operating on Bitcoin has created a critical infrastructure challenge: how can clients reliably discover and connect to protocol nodes without relying on centralized services? Traditional blockchain infrastructure platforms (such as Infura) represent single points of failure—a compromise or outage cascades to dependent applications. The Bitcoin ethos demands decentralized, trustless infrastructure where no single entity can censor, redirect, or corrupt client-to-node communication.

Simultaneously, protocols operating outside Bitcoin's native layer must solve the economic incentive problem: how do indexers and sequencers receive compensation for providing protocol services? Existing solutions typically employ either (a) separate token economies that introduce additional security assumptions, or (b) centralized reward distribution controlled by a single entity. Both approaches contradict the decentralization principles that motivate Bitcoin-native infrastructure.

The ACME Protocol addresses these challenges through two integrated systems. First, a decentralized DNS routing infrastructure uses Bitcoin's immutability to create an objective registry of verified nodes, with periodic on-chain commitments serving as cryptographic proof of correctness. Second, a purely Bitcoin-native fee distribution mechanism leverages script-enforced outputs and threshold cryptography to distribute rewards without requiring external validators or token systems. Together, these components create a self-reinforcing infrastructure ecosystem where trustworthiness is mathematically verifiable and economically aligned.

1.2 Technical Contributions

This work makes three primary technical contributions:

1. **Decentralized Node Verification and Discovery Architecture**: We present a hybrid approach combining blockchain-based name resolution with peer-to-peer discovery networks, enabling clients to identify and connect to nodes whose correctness is mathematically proven through Bitcoin-anchored commitments. This architecture eliminates centralized DNS operators while preserving the user experience of traditional domain-based services.

2. **Bitcoin-Native Trustless Fee Aggregation and Distribution**: We demonstrate how Bitcoin's scripting capabilities (hashlocks, timelocks, threshold signatures, and absolute/relative time verification opcodes) can implement complex conditional payment logic equivalent to smart contract functionality, enabling fully trustless reward distribution without altcoins or off-chain trust.

3. **Integration of Periodic Validation with Real-Time Health Monitoring**: We establish a dual-layer quality assurance mechanism where periodic on-chain commitments provide long-term correctness guarantees while real-time health checks ensure immediate responsiveness, creating a system that addresses both safety and liveness requirements.

---

2. System Overview and Architecture

2.1 Conceptual Framework

The ACME Protocol infrastructure operates across three distinct layers, each serving a specific function:

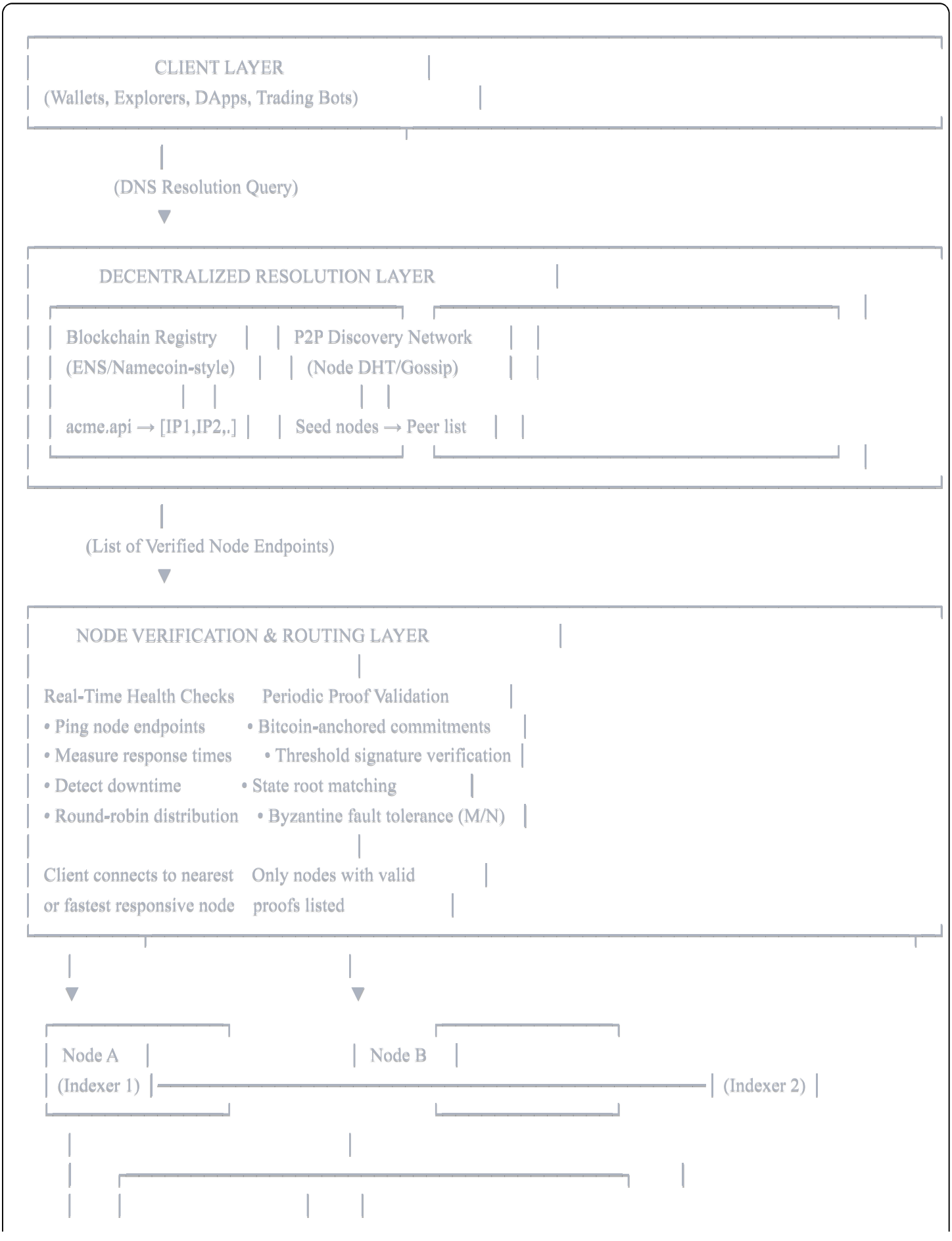
Layer 1 - Bitcoin Blockchain: Serves as the immutable settlement and verification layer. All state commitments, reward distributions, and critical coordination events are permanently recorded on Bitcoin, creating an objective truth source that cannot be forked or falsified without rewriting Bitcoin's history.

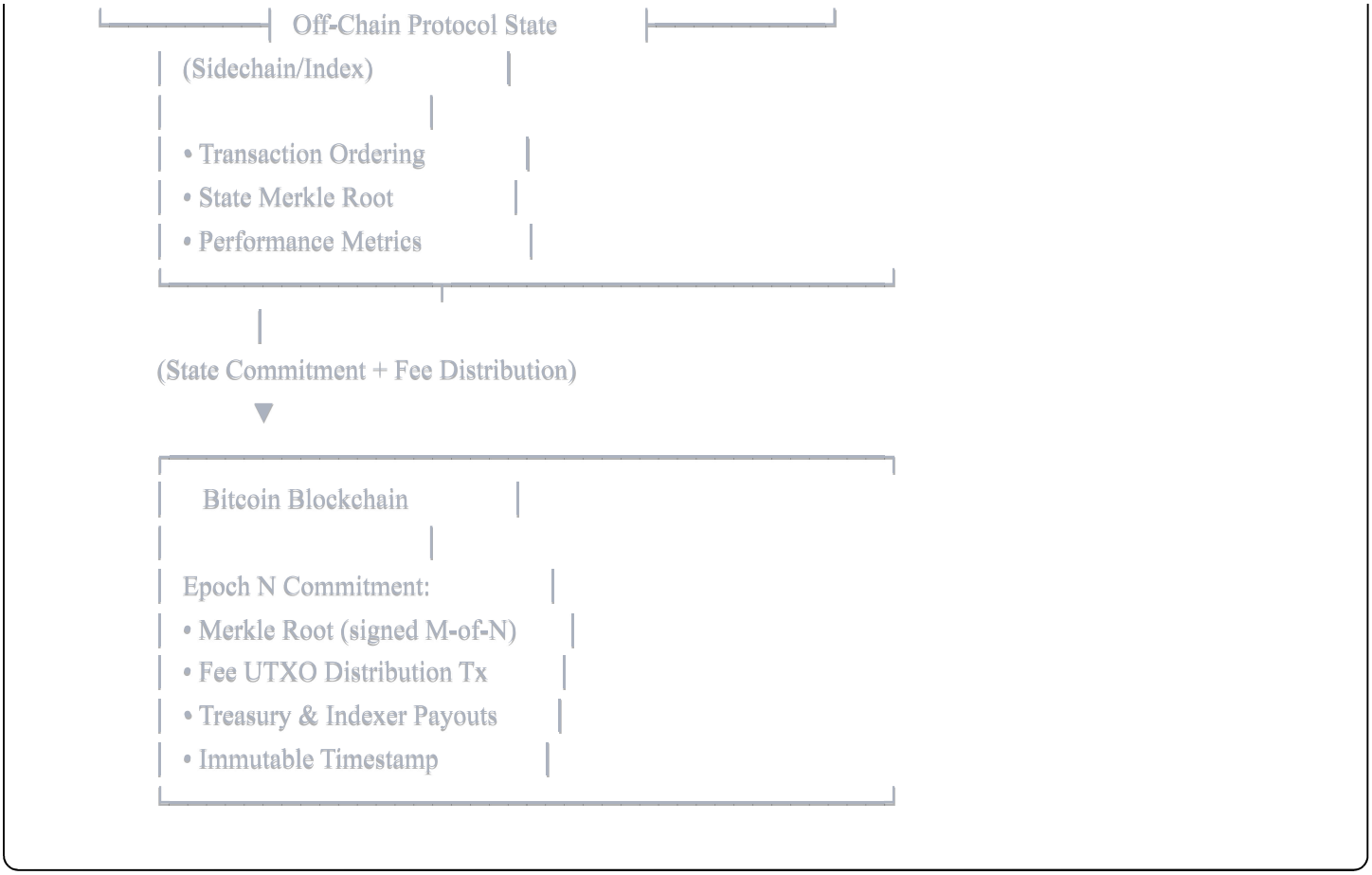
Layer 2 - Off-Chain Protocol State: The actual protocol execution occurs off-chain (in a sidechain, rollup, or index), maintaining transaction ordering, state mutations, and user-facing operations. This layer benefits from Bitcoin's security guarantees through periodic commitments to the base layer.

Layer 3 - Node Network: Indexers and sequencers run protocol nodes that (a) participate in off-chain state finalization, (b) prove their correctness through on-chain commitments, and (c) serve API requests to clients.

These nodes collectively maintain the protocol's availability and performance.

2.2 High-Level Architecture Diagram





---

3. Decentralized DNS Routing System

3.1 Design Rationale

Traditional DNS relies on a hierarchical, centralized system where a small number of authoritative nameservers control domain resolution. For critical blockchain infrastructure, this architecture presents several vulnerabilities:

- **Single Point of Failure**: DNS service provider outages immediately cascade to all dependent protocols and applications
- **Censorship Risk**: Regulatory pressure on DNS operators can block entire protocols or jurisdictions from accessing services
- **Man-in-the-Middle Vulnerability**: Compromised DNS infrastructure can redirect clients to malicious nodes serving incorrect data
- **Vendor Lock-In**: Protocol developers become dependent on infrastructure providers' policies and pricing

The ACME DNS routing system eliminates these vectors by distributing both the registry (which nodes are approved) and the resolution (how clients discover them) across a network of independent entities. The verification mechanism uses Bitcoin's consensus as an objective truth source: only nodes that can cryptographically prove their state correctness are advertised to clients.

3.2 Blockchain-Based Name Registry (Primary Approach)

3.2.1 Architecture and Implementation

The blockchain name registry leverages existing decentralized naming systems (such as Ethereum Name Service or Bitcoin-anchored Namecoin-style registries) to maintain a mapping from a stable domain name (e.g., `acme.api`) to a dynamically updated list of verified node endpoints.

The registry record for ACME contains:

```
Domain: acme.api
Owners: [Threshold Group Public Keys M-of-N]
Endpoints: [
  {
    address: "192.0.2.1:8080",
    pubkey: <IndexerAPublicKey>,
    proofHeight: 850000,
    weight: 1.0
  },
  {
    address: "203.0.113.45:8080",
    pubkey: <IndexerBPublicKey>,
    proofHeight: 850000,
    weight: 0.95
  },
  ...
]
UpdateTimestamp: 1731024000
CurrentEpoch: 127
```

Critical Design Feature: Registry updates require cryptographic authorization from the threshold group (e.g., 5 of 8 indexers). This ensures no single entity can manipulate the available endpoint list. The update process follows the epoch-based architecture described in Section 4: at each epoch boundary, indexers that successfully participated in state commitment become eligible for inclusion in the registry.

3.2.2 Client Resolution Process

A client application (wallet, explorer, DApp) performs the following sequence to resolve ACME's API endpoints:

1. **Query the Registry**: Using a standard DNS client library or blockchain RPC call, the client queries for the `acme.api` domain. If using ENS, this might resolve through `eth.limo` gateway or a local ENS resolver. If using Bitcoin-anchored system, a SPV client or light client queries the authoritative source.

2. **Verify Cryptographic Signature**: The returned record includes a threshold signature from the indexer group. The client verifies this signature using the known group public key. If verification fails, the record is rejected as potentially tampered.
3. **Extract Endpoint List**: The record contains multiple endpoints. The client extracts all endpoints marked as "active" and matching the current epoch.
4. **Select Endpoint**: Using strategies described in Section 3.5 (load balancing), the client selects one or more endpoints. This might involve geographic proximity, response time measurement, or random rotation.
5. **Validate Node Correctness**: Before accepting responses from the selected endpoint, the client performs verification described in Section 3.3.

Pseudocode: Client-Side Resolution

```pseudocode
```

```
function resolveAcmeEndpoints():
    record ← queryBlockchain("acme.api")

    if not verifyThresholdSignature(record.signature, GroupPubKey):
        RAISE InvalidSignature

    currentEpoch ← record.currentEpoch
    endpoints ← []

    for each endpoint in record.endpoints:
        if endpoint.proofHeight >= lastValidCheckpoint():
            endpoints.append(endpoint)

    if endpoints.isEmpty():
        RAISE NoValidEndpoints

    return endpoints


function selectEndpoint(endpoints):
    // Strategy: measure latency to all endpoints
    latencies ← {}
    for each endpoint in endpoints:
        latencies[endpoint] ← measurePing(endpoint)

    fastestEndpoint ← endpoints with minimum latency
    return fastestEndpoint


function validateNodeData(endpoint, response):
    // Verify response includes proof linking to committed state root
    lastCommitment ← getLastAnchoredStateRoot()
    proof ← extractMerkleProof(response)

    if not verifyMerkleProof(proof, response.data, lastCommitment):
        RAISE InvalidProof

    return True
```

3.3 On-Chain Commitments and Proof of Correctness

3.3.1 Checkpoint Architecture

At regular intervals (defined by protocol parameters, typically every 100 Bitcoin blocks or approximately 16 hours), the ACME indexer network reaches consensus on the current state and commits this state to Bitcoin. This periodic commitment serves as an objective checkpoint against which node correctness can be verified.

Commitment Structure:

```
BitcoinTransaction {
  inputs: [
    {
      txid: <previousEpochDistributionTx>,
      vout: 0,
      scriptSig: <thresholdSignature>
    }
  ],
  outputs: [
    {
      // State Commitment Output
      value: 0,
      scriptPubKey: OP_RETURN <epochNumber> <merkleRoot> <commitmentData>
    },
    {
      // Indexer Reward Output (example: HTLC)
      value: <rewardAmount>,
      scriptPubKey: <HTLC_SCRIPT>
    },
    {
      // Treasury Output
      value: <treasuryShare>,
      scriptPubKey: <treasuryAddress>
    }
  ],
  locktime: <epochEndHeight>
}
```

The `<merkleRoot>` is computed as the root of a binary Merkle tree containing:

- All validated transactions in the epoch
- State transitions for each asset/user
- Performance metrics for indexers
- Identity information for reward winners

This root serves as a cryptographic digest of the epoch's canonical state.

3.3.2 Threshold Signature Protocol

The commitment transaction is created through a distributed threshold signing protocol. The indexer group (defined by protocol governance) produces a single aggregated public key using schemes such as MuSig2 or FROST (Flexible Round-Optimized Schnorr Threshold signatures).

Signing Process:

1. **State Computation**: All honest indexers independently compute the same state root using identical deterministic logic applied to the same transactions.
2. **Round 1 - Commitment Phase**: Each indexer generates random nonces and sends nonce commitments to peers. These commitments prove participation without revealing nonce values.
3. **Round 2 - Challenge Phase**: The group collectively computes a challenge value based on the message (state root) and the aggregated nonce commitments.
4. **Round 3 - Response Phase**: Each indexer computes their response contribution using their private key, the challenge, and their nonce. The aggregation function combines these individual responses into one threshold signature.
5. **Publication**: The valid threshold signature (which requires at least M of N indexers to have honestly participated) is published in the Bitcoin transaction.

Cryptographic Guarantee: A threshold signature of M-of-N is only valid if at least M distinct participants collaborated honestly. It is computationally infeasible to forge without compromising M private keys. This directly implements Byzantine fault tolerance: if the threshold is set to 5-of-8, the system tolerates up to 3 Byzantine (malicious or failed) participants.

Mathematical Formulation:

For a FROST threshold signature scheme with threshold M and group size N:

- Aggregated public key: $PK\_agg = H(L, i) * PK\_i + H(L, j) * PK\_j + ...$ (Schnorr key aggregation)
- Each participant's response: $r\_i = s\_i + c * a\_i$ (where s_i is nonce contribution, c is challenge, a_i is private key share)
- Aggregated signature: $\sigma = \Sigma(r\_i)$ for the M participants
- Verification: $[\sigma]G = [R] + [c]PK\_agg$ where R is the aggregated nonce

The security reduces to the discrete logarithm problem: forging such a signature would require solving ECDLP, which is computationally intractable with current technology.

3.4 Dual Verification: Periodic Validation vs Real-Time Monitoring

3.4.1 Periodic Validation Layer (Safety)

The on-chain commitments provide periodic, cryptographically certain verification that a node's claimed state is correct as of a specific checkpoint. This addresses the "safety" property in distributed systems terminology: the guarantee that nothing bad (data corruption, state divergence) has occurred.

Validation Flow:

After each checkpoint commitment is confirmed on Bitcoin, the resolution system updates its registry:

```
Upon observing new commitment at block height B:

  1. Extract state root S_new and threshold signature sig

  2. Verify sig using Group Public Key
    if sig.verify(S_new, GroupPK) == False:
      LOG_ERROR("Invalid commitment")
      continue

  3. Extract participating indexers from commitment metadata
    participatingIndexers ← sig.extractParticipants()

  4. Update registry:
    for each indexer in participatingIndexers:
      registry[indexer].lastValidProof ← B
      registry[indexer].stateRoot ← S_new
      registry[indexer].valid ← True

  5. Update TTL for registry records:
    registryTTL ← B + 100_blocks  // Next checkpoint
```

Nodes that failed to participate in the commitment (perhaps due to being out of sync or offline) are temporarily marked as unverified until they catch up and participate in a future checkpoint.

3.4.2 Real-Time Health Monitoring Layer (Liveness)

Parallel to periodic validation, a real-time health monitoring system ensures that registered nodes are actually online and responsive at the moment clients need them. This addresses "liveness": the guarantee that services are actually accessible.

Health Check Mechanism:

Each node exposes a lightweight health endpoint (e.g., `GET /health`):

```
json
```

```
{
  "status": "healthy",
  "dbCaughtUp": true,
  "lastBlockHeight": 850000,
  "requestsPerSecond": 145.3,
  "p95Latency": 23,
  "uptime": 99.97
}
```

Health monitors (which could be independent community members, the node operators themselves, or dedicated monitoring services) periodically query this endpoint:

pseudocode

```
function healthCheckLoop():
    for each node in ActiveRegistry:
        health ← queryEndpoint(node.address + "/health", timeout=2s)

        if health == TIMEOUT:
            UpdateNodeMetrics(node, status=UNRESPONSIVE, latency=TIMEOUT)
        else if health.dbCaughtUp == false:
            UpdateNodeMetrics(node, status=SYNCING)
        else if health.status == "healthy":
            UpdateNodeMetrics(node,
                status=HEALTHY,
                latency=health.p95Latency)
        else:
            UpdateNodeMetrics(node, status=DEGRADED)

        wait(randomInterval(30s, 90s))
```

Metrics Aggregation: The collected health metrics are aggregated into a distributed reputation system. Nodes with consistent uptime and low latency receive higher weights in load balancing decisions, while nodes with frequent outages or high latencies are deprioritized.

3.4.3 Combined Effect

The two layers work synergistically:

- **Periodic validation** answers: "Is this node's data correct?" (verified once per epoch, cryptographically certain)
- **Real-time monitoring** answers: "Can I reach this node right now?" (checked continuously, heuristic but current)

A client making a request first verifies that the node has passed the most recent checkpoint (safety), then measures response time and success rate (liveness). The combination ensures both correctness and availability.

Example Timeline:

```
Time  Checkpoint  Node A Status          Node B Status
────  ──────────  ──────────────────────  ──────────────────────
T0    #126        Valid + Healthy        Valid + Healthy
T1    -           Valid + Latency: 15ms  Valid + Latency: 800ms
T2    -           Valid + Unresponsive   Valid + Healthy
T3    #127        ❌ Did Not Participate  ✓ Participated & Valid
T4    -           N/A (Not in registry)  Valid + Healthy

At T3, despite Node A having been healthy, it failed to
participate in checkpoint. Clients will not route traffic
to Node A until it proves correctness in checkpoint #128.
```

3.5 Load Balancing and Networking Strategies

3.5.1 Decentralized DNS Load Balancing

Traditional load balancers are centralized bottlenecks. ACME uses DNS-level load balancing where the registry returns multiple addresses with controlled ordering:

Multi-Answer DNS Resolution:

```
Query: acme.api
Response: [
  {address: "192.0.2.1", weight: 100, region: "us-east"},
  {address: "203.0.113.45", weight: 90, region: "eu-west"},
  {address: "198.51.100.1", weight: 80, region: "ap-south"}
]
```

Clients receiving multiple addresses distribute requests among them. The registry can rotate the ordering of responses (DNS servers traditionally rotate the order of returned answers), providing implicit load distribution without any central coordinator.

Weighted Distribution Algorithm:

The registry uses endpoint weights to proportionally distribute traffic:

```pseudocode
```

```
function selectEndpointWeighted(endpoints):
  totalWeight ← sum(e.weight for e in endpoints)
  randomValue ← random(0, totalWeight)

  cumulativeWeight ← 0
  for each endpoint in endpoints:
    cumulativeWeight += endpoint.weight
    if randomValue <= cumulativeWeight:
      return endpoint
```

Weights are dynamically adjusted based on reported health metrics. An endpoint showing high latency or frequent errors has its weight reduced, naturally shifting traffic to better performers.

3.5.2 Geographic Distribution and Anycast

For optimal performance, clients should connect to nodes with low network latency. The registry can support region-specific domains:

```
us.acme.api   → [IPs of US-based indexers]
eu.acme.api   → [IPs of EU-based indexers]
ap.acme.api   → [IPs of Asia-Pacific-based indexers]
acme.api      → [All IPs, globally distributed]
```

A sophisticated client implementation could detect its geographic region (through IP geolocation or explicit configuration) and query the appropriate regional domain. Advanced implementations might even pre-emptively measure latency to multiple regions' endpoints and select the fastest.

Anycast Networking (Optional Enhancement):

For deployments where indexer operators can coordinate network infrastructure, Anycast provides automatic geographic routing at the BGP layer. In Anycast:

- All nodes advertise the same IP address (e.g., 192.0.2.0/24) from their respective networks
- BGP routing naturally directs client traffic to the topologically nearest node announcing that IP
- If a node goes offline, BGP stops advertising, traffic shifts to the next closest node automatically

This eliminates the need for the client to make routing decisions—the Internet's routing infrastructure handles it automatically. However, Anycast requires coordination and is optional; the DNS-based approach works perfectly well for decentralized scenarios where node operators don't coordinate infrastructure.

3.5.3 Client-Side Failover and Redundancy

To further improve reliability, clients implement sophisticated failover strategies:

```pseudocode
function robustApiRequest(method, params):
  endpoints ← resolveAcmeEndpoints()

  // Try primary endpoint
  try:
    response ← callEndpoint(endpoints[0], method, params, timeout=2s)
    if response.isValid():
      return response
  catch TimeoutException:
    // Fall through to failover

  // Try secondary endpoints in parallel
  for i = 1 to min(3, endpoints.length):
    spawn_async {
      try:
        response ← callEndpoint(endpoints[i], method, params, timeout=3s)
        if response.isValid():
          return response
      catch:
        continue
    }

  // If all return, use fastest
  responses ← [collect all responses up to 3s total]
  if not responses.isEmpty():
    return responses[0]  // First to respond

  RAISE AllEndpointsFailed
```

This "multi-homing" approach treats all endpoints as substitutes. Since all nodes have identical data (by design and verification), the client is indifferent between them—it simply chooses based on performance.

---

4. Bitcoin-Native Fee Distribution Architecture

4.1 Design Principles and Economic Rationale

The fee distribution system must satisfy several competing requirements:

1. **Trustlessness**: No central party should be able to embezzle funds or incorrectly allocate rewards. The system must be enforced by Bitcoin's consensus rules, not off-chain agreements.

2. **Decentralization**: No single indexer should control the distribution mechanism. Decisions about who receives payment must require threshold agreement.
3. **Efficiency**: The system should minimize on-chain transactions and fees while remaining practical for regular distributions.
4. **Fairness**: The allocation mechanism must be transparent, predictable, and resistant to manipulation by any subset of participants.
5. **Fallback Safety**: If participants disappear or fail to cooperate, funds should never be permanently locked. Treasury fallbacks ensure capital recovery.

These principles guide away from naive solutions (centralized coordinator, altcoin tokens, or weak governance) toward a pure Bitcoin script-based mechanism.

4.2 Fee Collection Phase: Aggregation and Timelocks

4.2.1 Aggregation UTXO Structure

During each epoch, users pay fees directly to Bitcoin addresses controlled by a script that locks funds until epoch conclusion. This script is constructed as follows:

```bitcoin
# Aggregation UTXO Redeem Script
<epoch_end_block_height> OP_CHECKLOCKTIMEVERIFY OP_DROP
<indexer_group_aggregate_pubkey> OP_CHECKSIG
```

Script Semantics:

- `OP_CHECKLOCKTIMEVERIFY <height>`: Fails if the transaction's lock time is less than `<height>`. In other words, the transaction is only valid if it's being confirmed at or after the specified block height.
- `OP_DROP`: Removes the lock time value from the stack, leaving only the public key for signature verification
- `OP_CHECKSIG`: Requires a valid signature from the group's aggregate public key

Transaction Lifecycle:

```
Epoch Start (Block 1000):
  Genesis Tx creates aggregation UTXO with script above (height = 1100)
  UTXO is "frozen"—cannot be spent regardless of signature

Blocks 1000-1099:
  Users send fees to aggregation address: 0.001 BTC, 0.0005 BTC, ...
  All inputs consolidate into single UTXO (or are tracked separately)
  Accumulation: total becomes 0.5 BTC after 100 blocks

Block 1100 (Epoch End):
  Timelock condition finally becomes satisfiable
  Indexer group can now create distribution transaction

Block 1100+:
  Distribution transaction spends the UTXO
  Commitment is anchored, rewards are allocated
  Previous epoch's distribution Tx creates next epoch's aggregation UTXO
  Cycle repeats
```

4.2.2 Threshold Control Mechanism

The aggregation UTXO requires a threshold-aggregated public key (`<indexer_group_aggregate_pubkey>`) to unlock. This public key is derived through Schnorr key aggregation from N indexers' individual public keys:

MuSig2 Key Aggregation:

Each indexer `i` has a private key `x_i` and corresponding public key `X_i = [x_i]G`.

The group's aggregate public key is computed as:

```
L = hash(X_1 || X_2 || ... || X_N)  // Rogue key prevention
a_i = hash(L || X_i)                // Individual tweak
X_agg = [a_1]X_1 + [a_2]X_2 + ... + [a_N]X_N
```

The tweaks `a_i` prevent "rogue key attacks" where a malicious indexer could choose their public key to cancel out other participants' contributions.

To spend the aggregation UTXO, at least M of the N indexers must cooperate to produce a valid Schnorr signature over the distribution transaction. An individual indexer cannot produce a valid signature alone, and a group of fewer than M indexers cannot produce one through any combination of their private keys.

4.3 State Commitment and Anchoring

4.3.1 Merkle Root Construction

At epoch end, indexers compute a Merkle root representing all canonical state:

```
Layer 0 (Leaf nodes):
  Leaf[0] = hash(Transaction #0)
  Leaf[1] = hash(Transaction #1)
  ...
  Leaf[N-1] = hash(Transaction #N-1)

Layer 1:
  Node[0] = hash(Leaf[0] || Leaf[1])
  Node[1] = hash(Leaf[2] || Leaf[3])
  ...

... (continue until single root)

Root = MerkleRoot
```

This root serves as a compact, deterministic commitment to the exact set of transactions and their ordering. Any change to the transaction set, order, or content produces a different root, making the commitment tamper-evident.

4.3.2 Anchor Transaction Structure

The commitment is embedded in a Bitcoin transaction with the following structure:

```
bitcoin
```

```
{
 "vin": [
  {
   "txid": "previous_epoch_distribution_txid",
   "vout": 0,
   "scriptSig": "<threshold_signature_from_M_of_N_indexers>"
  }
 ],
 "vout": [
  {
   "value": 0,
   "scriptPubKey": "OP_RETURN <epoch_id><merkle_root><commitment_metadata>"
  },
  {
   "value": 0.25,
   "scriptPubKey": "<HTLC_SCRIPT_FOR_INDEXER_A>"
  },
  {
   "value": 0.25,
   "scriptPubKey": "<TREASURY_ADDRESS>"
  }
 ]
}
```

The `OP_RETURN` output carries the commitment data (epoch number, Merkle root, and optional metadata). `OP_RETURN` outputs are provably unspendable—they can never be spent as inputs to future transactions, making them permanent data carriers on Bitcoin's ledger.

Critical Property: Because this transaction is part of the Bitcoin blockchain, its:

- **Existence** is guaranteed (anyone can verify it was mined)
- **Immutability** is guaranteed (rewrites would require rewriting Bitcoin's proof-of-work, computationally infeasible)
- **Ordering** is guaranteed (block heights provide a total ordering of all transactions)
- **Timestamp** is approximately known (block creation times)

These properties make Bitcoin's ledger an objective repository of protocol state history.

4.4 Trustless Reward Distribution Mechanisms

The distribution transaction can allocate fees through two different mechanisms, each with different trustlessness guarantees and efficiency characteristics.

4.4.1 Option A: Direct Payout

In the direct payout approach, the distribution transaction immediately pays out rewards to designated recipients:

```
bitcoin

{
 "vout": [
  {"value": 0, "scriptPubKey": "OP_RETURN <merkle_root>"},

  // Indexer reward (50% of fees)
  {"value": 0.25, "scriptPubKey": "OP_WPKH <indexer_a_pubkey_hash>"},

  // Treasury (50% of fees)
  {"value": 0.25, "scriptPubKey": "OP_WPKH <treasury_multisig_hash>"},

  // Miner fee (implicit)
  // input_total - sum(outputs) = fee
 ]
}
```

As soon as the distribution transaction confirms, the indexer can spend their output to their own wallet using a standard P2WPKH spend.

Trust Model: This mechanism trusts the threshold signing group to correctly compute and allocate rewards according to protocol rules. If a malicious majority (M-1 or more colluding indexers) control the signature, they could create incorrect outputs (e.g., paying themselves extra, omitting treasury, paying wrong recipients).

However, this malicious behavior would be **immediately visible on-chain** and violate the protocol's social contract. Detection would likely trigger community responses: reputation damage, slashing on the sidechain level, economic penalties, or exclusion from future participation.

Advantages:

- Minimal on-chain complexity (only 1 transaction)
- Immediate reward settlement
- Low fees (small transaction size)

Disadvantages:

- Relies on social enforcement rather than cryptographic enforcement
- Malicious majority could steal

4.4.2 Option B: Hash-Time Locked Contracts (HTLCs)

To add cryptographic enforcement, rewards can be distributed through HTLCs, a technique borrowed from Lightning Network payment channels and atomic swaps:

HTLC Redeem Script:

```
bitcoin

OP_IF
  OP_HASH160
  <hash_of_secret_R>
  OP_EQUALVERIFY
  <indexer_pubkey>
  OP_CHECKSIG
OP_ELSE
  <timelock_in_blocks>
  OP_CHECKLOCKTIMEVERIFY
  OP_DROP
  <treasury_pubkey>
  OP_CHECKSIG
OP_ENDIF
```

Script Logic:

- **IF branch** (hashlock path): If the indexer reveals a secret whose hash matches `<hash_of_secret_R>` AND provides their signature, they can spend the output immediately.
- **ELSE branch** (timelock/fallback path): After `<timelock_in_blocks>` passes without the secret being revealed, the treasury can spend the output with just their signature.

The two-step process works as follows:

Step 1 - Commitment Creation: The distribution transaction creates HTLC outputs for each indexer reward:

```
Distribution Tx (Block 1101):
  Input: 0.5 BTC from aggregation UTXO
  Outputs:
    - OP_RETURN <merkle_root>
    - HTLC (0.25 BTC): Redeemable by indexer A with secret or treasury after 100 blocks
    - Treasury (0.25 BTC): Direct to treasury address
```

Step 2 - Reward Claim: If the indexer is online and responsive, they create a claim transaction spending the HTLC output:

```
Claim Tx (Block 1102, created by Indexer A):
  Input: HTLC output (0.25 BTC)
  Unlock script: <signature> <secret_R> 1
  Output: 0.249 BTC to Indexer A's address
  (0.001 BTC paid as transaction fee)
```

By providing the secret, the indexer proves they are the rightful recipient (they pre-committed the hash at epoch start). The secret's revelation on-chain makes the commitment publicly visible but is one-time-use (the same secret won't help anyone else claim a different HTLC).

Step 3 - Fallback: If the indexer disappears or chooses not to claim, the treasury fallback becomes available after the timelock:

```
Fallback Tx (Block 1201, created by Treasury):
  Input: HTLC output (0.25 BTC)
  Unlock script: <treasury_sig> (no secret, uses ELSE branch)
  nLockTime: >= 1201 (satisfies OP_CHECKLOCKTIMEVERIFY 100)
  Output: 0.249 BTC to Treasury address
```

Advantages:

- Pure cryptographic enforcement: only the rightful holder of secret+key can claim before timelock
- No trust in signers' integrity needed
- Malicious indexers gain nothing by trying to claim others' HTLCs (they don't have the secrets)
- Treasury is guaranteed to recover funds eventually

Disadvantages:

- Requires 2 transactions per indexer (claim tx), approximately doubling on-chain overhead
- More complex script logic
- Higher total fees if many indexers per epoch

4.4.3 Comparative Analysis

The choice between mechanisms depends on deployment context:

Property	Direct Payout	HTLC
Trustlessness	Social	Cryptographic
Transactions	1	1 + N (N = # winners)
Total size	~200 bytes + outputs	~200 bytes + HTLC outputs + N claims
Average fee cost	0.001-0.002 BTC	0.005-0.01 BTC (if N ≥ 3)
Settlement latency	1 confirmation	2 confirmations (claim tx)
Failure mode	Malicious payout	Unclaimed funds revert to treasury
Suitable for	High-trust teams	Decentralized, adversarial settings

For ACME's decentralized context, the HTLC approach provides stronger guarantees but at moderate cost. Many protocols would find this cost-benefit tradeoff acceptable given the additional security.

4.5 Reward Calculation and Allocation

4.5.1 Single-Winner Model

In each epoch, the protocol can designate a single winning indexer who receives all indexer rewards. The winner might be determined by:

- **Leader election**: A pre-agreed index into a pseudo-random sequence determines the leader for epoch N
- **Performance-based**: The indexer with best uptime/latency metrics wins
- **Lottery**: Random selection with probability weighted by stake or contribution

The distribution transaction outputs are simply:

```
Outputs:
  - OP_RETURN <commitment>
  - P2WPKH to Winner (0.25 BTC)
  - P2WPKH to Treasury (0.25 BTC)
```

Script Complexity: Minimal. Verification is straightforward: did the right person/address receive the right amount?

4.5.2 Multi-Recipient Model

If the protocol wants to reward multiple indexers (e.g., top 3 performers, or all participants equally), the distribution transaction includes multiple outputs:

```
Single-Winner Distribution:
  outputs:
    - OP_RETURN
    - [0.25 BTC to Winner]
    - [0.25 BTC to Treasury]

Multi-Recipient Distribution (3 winners):
  outputs:
    - OP_RETURN
    - [0.15 BTC to IndexerA]
    - [0.08 BTC to IndexerB]
    - [0.02 BTC to IndexerC]
    - [0.25 BTC to Treasury]
```

Multi-recipient designs encourage broader participation but increase script complexity proportionally to the number of recipients.

Scaling Consideration: Bitcoin transactions have a ~100 KB size limit, which means a theoretical maximum of roughly 1,000-2,000 outputs (each ~40-60 bytes). Practical limits depend on other data. If ACME wants to reward more recipients per epoch, it could:

1. **Limit rewards per epoch**: Only reward top N indexers (e.g., $N \leq 10$)
2. **Aggregate across epochs**: Consolidate monthly/quarterly rewards into single payout
3. **Hybrid model**: On-chain commitments per epoch, but claim/distribution done off-chain (reduces trustlessness)

4.6 Treasury Distribution and Fallback Logic

4.6.1 Treasury Direct Allocation

The treasury's share (typically 50% of fees) is allocated directly to a known treasury address (which could be a multi-sig address controlled by core developers, a DAO contract on another chain, etc.):

```
bitcoin

# Treasury Output
{
  "value": 0.25,
  "scriptPubKey": OP_DUP OP_HASH160 <treasury_addr_hash> OP_EQUALVERIFY OP_CHECKSIG
}
```

This is a standard P2PKH output. No timelock is necessary since the treasury is not conditional or contested.

4.6.2 Unclaimed Reward Fallback

If an indexer doesn't claim their HTLC reward before the timelock expires, the treasury automatically becomes eligible:

```
Timeline:
  Block 1101: Distribution TX confirmed, HTLC creates 0.25 BTC reward for IndexerA
  Block 1102-1200: IndexerA has 100 blocks (~16 hours) to claim
  Block 1200: Timelock expires, ELSE branch becomes valid
  Block 1201+: Treasury can claim the 0.25 BTC
```

This ensures no rewards are permanently locked. If an indexer goes offline, becomes unreachable, or loses their keys, the treasury recovers the funds for reallocation in future epochs or as protocol savings.

4.6.3 No-Valid-Commitment Scenario

If the indexers never produce a valid commitment transaction (perhaps due to network partition, Byzantine behavior, or complete failure), the fee pool UTXO itself should not lock funds permanently. The aggregation UTXO's script can include a secondary fallback:

bitcoin

```
# Enhanced Aggregation UTXO Script (with fallback)
OP_IF
   <epoch_end_block_height> OP_CHECKLOCKTIMEVERIFY OP_DROP
   <indexer_group_aggregate_pubkey> OP_CHECKSIG
OP_ELSE
   <fallback_height>  # epoch_end + 200 blocks (another ~32 hours)
   OP_CHECKLOCKTIMEVERIFY OP_DROP
   <treasury_pubkey> OP_CHECKSIG
OP_ENDIF
```

With this dual timelock, if indexers fail to produce a valid commitment by epoch end + 200 blocks, the treasury can directly claim the entire fee pool as an emergency recovery mechanism.

---

5. Integrated Flow and Examples

5.1 Complete Epoch Lifecycle

This section traces a complete epoch through all stages of commitment and distribution.

Epoch Timeline Example (simplified, all values in BTC):

PREPARATION PHASE (Block 800)

Governance determines:

- Epoch 100 runs blocks 800-900
- Reward distribution: 50% indexers, 50% treasury
- Single winner (highest uptime)
- Winner reward: 0.5 BTC
- Indexer group: 8 indexers, threshold 5-of-8

FEE COLLECTION PHASE (Blocks 801-900, ~3.3 hours)

Aggregation UTXO created (block 800):

Script: 900 CLTV + aggreg_pubkey CHECKSIG

Cannot be spent until block 900

Users pay protocol fees:

Block 801: User A sends 0.01 BTC to aggregation addr

Block 815: User B sends 0.005 BTC to aggregation addr

Block 850: User C sends 0.02 BTC to aggregation addr

... (100 transactions across 100 blocks)

Block 900: Total accumulated = 0.5 BTC

All payments are to the same script address

(could be consolidated into single UTXO or tracked separately)

STATE FINALIZATION PHASE (Block 900)

Indexers compute canonical state:

- All 100 user transactions ordered and included
- State Merkle root = abc123... (32 bytes)
- Performance metrics compiled
- Winner (Indexer #3) identified

Threshold signing group consensus:

- Indexer 1: Signs root hash ✓
- Indexer 2: Signs root hash ✓
- Indexer 3: (winner) Signs distribution ✓
- Indexer 4: Signs root hash ✓
- Indexer 5: Offline ✗
- Indexer 6: Signs root hash ✓
- Indexer 7: Delayed ✗
- Indexer 8: Signs root hash ✓

Result: 6 of 8 signed (threshold 5 met) → Valid!

```
COMMITMENT TRANSACTION BROADCAST (Block 901)
  Transaction constructed:
    Input: 0.5 BTC from aggregation UTXO
    Signature: Aggregated Schnorr sig from 6 indexers

    Output 1 (OP_RETURN):
      Data: Epoch#100 | Root:abc123... | Timestamp:920000000
      Value: 0

    Output 2 (Indexer #3 Reward - HTLC):
      Script: IF secret+sig ELSE 100-block-timeout+treasury-sig ENDIF
      Value: 0.25 BTC
      Hash: hash(pre-committed-secret-for-idx3)

    Output 3 (Treasury):
      Address: Treasury P2WPKH
      Value: 0.25 BTC

    Fee: 0.0001 BTC (implicit)

  Transaction broadcast to Bitcoin network
  Mined in block 901 (confirmation: 1)

REWARD CLAIM PHASE (Blocks 902-950)

  Block 902: Indexer #3 sees their HTLC is confirmed
    Creates claim transaction:
      Input: HTLC output (0.25 BTC)
      Unlock: <sig> <secret> 1
      Output: 0.249 BTC to their address
      Fee: 0.001 BTC

    Transaction included in block 902
    Indexer #3 successfully claimed 0.249 BTC

  Treasury immediately spends their output:
    Input: Treasury output (0.25 BTC)
    Output: 0.249 BTC to treasury address

    Treasury share secured

NEXT EPOCH INITIALIZATION (Block 951)
  From the same commitment tx (or triggered by it):
    New aggregation UTXO created for Epoch #101
```

Script: 1051 CLTV + aggreg_pubkey CHECKSIG
Fresh 100-block window begins
Fee collection resumes

5.2 Failure Scenario: Offline Indexer

SCENARIO: Indexer #3 goes offline before reward claim

PHASE TIMELINE:

Block 901: Commitment TX with Indexer #3's HTLC confirmed
Blocks 902-950: Indexer #3 offline (server crashed)
Block 1001: Timelock expires (100 blocks passed)

TREASURY RECOVERY:

At Block 1001+, Treasury can spend the HTLC:
Input: HTLC output from Block 901
Unlock script: <treasury_sig> (ELSE branch, no secret needed)
nLockTime: 1001 (satisfies CLTV 100)
Output: 0.249 BTC to treasury

Result:
- Indexer #3 loses their reward (offline = no participation benefit)
- Treasury recovers the 0.25 BTC
- Capital not permanently locked
- Next epoch begins normally

PREVENTION/IMPROVEMENT:
- Indexer #3 should monitor their addresses
- Could automate claim with key management service
- Could claim preemptively immediately after confirmation
- Higher timeout value (200+ blocks) gives more reaction time

5.3 Byzantine Attack Scenario: Collusion

SCENARIO: 5 of 8 indexers collude to steal treasury share

ATTACK ATTEMPT:

Colluding group (Indexers 1,2,3,5,8) creates forged commitment:
Output 1: OP_RETURN with valid root ✓
Output 2: HTLC for Indexer #1 (winner): 0.4 BTC (overallocated)

Output 3: Treasury: 0.1 BTC (reduced)
Output 4: Indexer #2 extra bribe: 0.04 BTC

Signature attempt:
They have keys for 5 indexers (threshold is 5-of-8)
They CAN produce valid threshold signature for this transaction ✓

ATTACK SUCCEEDS AT BITCOIN LAYER:

Bitcoin consensus rules:
- Threshold signature: valid ✓
- Spending from aggregation UTXO: valid ✓
- Script execution: passes ✓
- Transaction: accepted into mempool, mined ✓

DETECTION AT PROTOCOL LAYER:

Block 901: Malicious distribution TX confirmed

Community monitoring:
- Developers see unexpected outputs
- Outputs differ from protocol rules:
  * Indexer #1 getting 0.4 instead of 0.25
  * Treasury getting 0.1 instead of 0.25
  * Unexpected output to Indexer #2

- Calculate: 5 of 8 indexers signed this
- Identify which 5 based on threshold signature math (off-chain analysis)

RESPONSE OPTIONS:

1. Social Recovery:
   - Community consensus identifies the 5 colluders
   - Their reputation damaged
   - Slashing on sidechain level (burn their stakes)
   - Removal from next epoch's signing group
   - Loss of future rewards

2. Fork Consideration:
   - If 5 of 8 represents majority, that IS majority consensus
   - Protocol community could choose to fork (social/political decision)
   - Bitcoin itself is neutral—doesn't choose sides

3. Governance Response:

```
  - Increase threshold to 6-of-8 (requires more consensus)
  - Rotate indexer set more frequently
  - Implement reputation/slashing more aggressively

LESSON:
 Bitcoin's consensus can enforce correct signatures/scripts
 But cannot enforce "intent" if signers are malicious
 This is why threshold is set above 50%:
  - 5-of-8 majority can do anything
  - Honest minority (3) cannot prevent it
  - But malicious majority is visible and accountable

 Pure trustlessness at Bitcoin layer would require:
  - All reward amounts pre-committed on-chain (not possible in general)
  - Or purely algorithmic winner selection (deterministic)
```

---

6. Efficiency Analysis

6.1 On-Chain Overhead

The periodic commitment approach amortizes Bitcoin fees across many protocol transactions. Consider an epoch with N user transactions:

Fee Cost Calculation:

```
Bytes per commitment TX:
 - Input: 108 bytes (SegWit witness sig/pubkey)
 - OP_RETURN output: 40-80 bytes (epoch# + root hash)
 - Reward outputs: 34 bytes each (P2WPKH)
 - Overhead: ~300 bytes total

Bitcoin fee rate: 20 satoshis/vB (typical for Bitcoin in Nov 2025)

Fee per commitment: 300 bytes × 20 sat/vB = 6,000 satoshis = 0.00006 BTC

User transactions per epoch: ~1,000 (typical)
Protocol revenue per epoch: 0.5 BTC in fees (hypothetical)

Overhead per user: 0.00006 BTC / 1,000 = 0.00000006 BTC

Cost per user transaction: User_fee + 0.00000006 BTC ≈ negligible increase
```

This demonstrates how batching via epochs makes periodic commitments economically viable.

6.2 Scalability Considerations

Scenarios and Implications:

1. **Very High Frequency (1,000s of epochs per year)**:
   - More frequent proofs (more secure)
   - Higher total Bitcoin fee overhead
   - Could aggregate multiple epochs per commitment if fees become excessive
2. **Large Indexer Set (50+ indexers)**:
   - Threshold signature still produces 64-byte aggregated sig (independent of N)
   - But coordination overhead increases with N
   - Could split into multiple threshold groups (each sub-group commits)
3. **Many Reward Recipients (100+ per epoch)**:
   - Transaction size increases proportionally
   - Could hit Bitcoin's ~100KB block size target
   - Solution: limit winners per epoch or use tiered distribution

6.3 Trade-offs Summary

Parameter	Benefit of Higher	Drawback of Higher
Epoch Length	Lower fees, more fee accumulation	Slower reward distribution, more network lag tolerance needed
Threshold (M/N)	Better Byzantine fault tolerance	Slower consensus, higher coordination cost
Reward Recipients	Fairer distribution	Larger transactions, higher fees
HTLC vs Direct	Better cryptographic guarantee	Extra on-chain transaction per recipient

---

7. Security Analysis

7.1 Threat Model

We consider an adversary that may:

- Control up to M-1 of N indexers (where M is the threshold)
- Operate malicious nodes advertising incorrect data
- Attempt to redirect client traffic to malicious endpoints
- Try to manipulate reward allocation

- Experience network partitions or Byzantine failures

The adversary cannot:

- Rewrite Bitcoin's history (assumption of Bitcoin security)
- Forge cryptographic signatures without private keys
- Compromise Bitcoin nodes' consensus rules

7.2 Security Guarantees by Component

DNS Routing Security:

1. **Endpoint List Authenticity**: The registry is cryptographically signed by M of N indexers. An attacker controlling M-1 indexers cannot forge an update. Each update includes a threshold signature verifiable against the known group public key.
2. **Endpoint Verification**: Only endpoints that have participated in recent checkpoints are advertised. A malicious node that hasn't proved correctness is not in the registry, so clients never connect to it.
3. **Temporary Node Failure Tolerance**: If a node goes offline (natural failure), it remains in registry until the next checkpoint (100 blocks later). Clients experience brief disruption when trying that endpoint, but failover to others occurs quickly. No permanent damage.
4. **Byzantine Tolerance**: If up to M-1 indexers are malicious and try to include a dishonest endpoint in the registry, they cannot—the update requires M signatures. Clients are protected from Byzantine nodes by the threshold requirement.

Fee Distribution Security:

1. **Fund Aggregation Integrity**: The timelock (CLTV) ensures fees cannot be prematurely withdrawn. Until the epoch end block, any signature is invalid.
2. **Threshold-Controlled Distribution**: Creating a valid distribution transaction requires M-of-N consensus. Fewer colluders cannot create a valid spend.
3. **Cryptographic Proof of Correctness**: Using HTLC with secrets ensures that only the rightful reward recipient (who knows the secret) can claim before timelock. Imposters cannot claim others' rewards.
4. **Treasury Fallback**: Unclaimed rewards revert to treasury after timelock, preventing permanent fund lockup.
5. **Visible on-Chain Cheating**: If M-1 honest indexers and their coalition somehow succeed in creating an invalid distribution, it's permanently recorded on Bitcoin for all to see. Off-chain slashing/reputation mechanisms activate.

7.3 Failure Modes and Mitigations

Failure Mode 1: Entire Indexer Set Goes Offline

Scenario: All 8 indexers crash simultaneously mid-epoch.

Consequences:

- No commitment transaction produced
- Fee pool remains locked by CLTV until fallback activates
- No state updates for this epoch

Mitigation:

- 200-block fallback timelock allows treasury to recover funds
- Governance can replace offline indexers with new ones
- Next epoch resumes with new indexer set

Failure Mode 2: Threshold Number of Indexers Become Byzantine

Scenario: Exactly M of N indexers are compromised and coordinate malicious actions.

Consequences:

- They CAN produce valid threshold signatures
- They CAN create invalid distributions
- Bitcoin consensus cannot distinguish valid from invalid (both pass cryptographic checks)

Mitigation:

- **Detection**: Malicious distribution is visible on-chain and observable by honest parties
- **Social Response**: Community removes compromised indexers for next epoch, implements governance punishment
- **Threshold Adjustment**: Governance increases threshold (e.g., from 5 to 6) to require supermajority
- **Monitoring**: Community runs monitoring nodes that watch for protocol violations and alert

Failure Mode 3: Network Partition

Scenario: Bitcoin network experiences partition; ACME indexers split into two groups.

Consequences:

- Each partition might try to produce its own commitment
- Two competing commitments could be mined

- Clients might see different versions

Mitigation:

- Bitcoin consensus has a canonical longest chain
- The commitment mined first (confirmed first) is canonical
- Later commitment is orphaned/reorganized away
- ACME clients follow Bitcoin's consensus (all see same "winning" commitment)
- Community governance resolves any persistent split

---

8. Comparison with Existing Approaches

8.1 Centralized Infrastructure (Infura Model)

Architecture: Single company operates nodes, clients connect to their APIs.

Advantages:

- Simple to deploy
- High performance coordination
- Transparent SLAs

Disadvantages:

- Single point of failure
- Regulatory and censorship vulnerabilities
- Misaligned incentives (infrastructure provider interests ≠ protocol interests)
- Represents extractive middleman

ACME Advantage: Eliminates central coordinator; distributes both infrastructure and governance.

8.2 Federated Architecture (Polkadot/Cosmos Model)

Architecture: Fixed set of validators operate infrastructure; clients connect to any validator.

Advantages:

- Decentralized infrastructure
- Clear validator set
- Simple validator selection

Disadvantages:

- Still permissioned (governance chooses validators)
- Potential validator capture (small set of powerful entities)
- Requires external blockchain (Polkadot parachain, Cosmos zone)

ACME Advantage: Permissionless (any indexer can join); uses Bitcoin for verification rather than external chain; no external token dependencies.

8.3 DAO Treasury Distribution Model

Architecture: Smart contract on Ethereum/other chain holds and distributes fees.

Advantages:

- Transparent on-chain logic
- Automated execution
- Clear rule enforcement

Disadvantages:

- Requires external blockchain (adds dependency)
- Subject to that chain's security and governance
- May require separate token for incentives
- Not truly Bitcoin-native

ACME Advantage: Uses only Bitcoin; no external dependencies; pure script enforcement; no token required.

---

9. Implementation Considerations

9.1 Key Management for Threshold Signing

Running threshold signature schemes in production requires careful key management:

1. **Key Generation**: Must use secure multi-party computation (MPC) to generate keys such that no party ever sees the complete private key.
2. **Key Storage**: Each indexer stores only their key share, preferably in hardware wallets or HSMs (Hardware Security Modules) with high availability.
3. **Backup and Recovery**: Key shares must be backed up securely; loss of M shares means loss of signing ability.

4. **Signing Coordinator**: Requires a coordinator (could be automated service or human) to orchestrate the threshold signing protocol among N participants.

9.2 Client Implementation

Clients implementing the DNS routing should:

1. **Cache Endpoints**: Store resolved endpoints locally with TTL matching epoch length for reduced lookup latency.
2. **Implement Fallover**: Have multiple endpoints loaded and ready to switch if primary fails.
3. **Validate Proofs**: For security-critical applications, verify Merkle proofs against the committed state root to ensure node data is correct.
4. **Monitor Health**: Ping endpoints periodically to update their perceived health/latency.

9.3 Monitoring and Alerts

A robust production ACME deployment should run:

1. **On-Chain Monitor**: Continuously watches Bitcoin blockchain for new ACME commitments; alerts on unusual patterns.
2. **Node Monitor**: Pings all registered endpoints; tracks uptime and latency; alerts on degradation.
3. **Signature Validator**: Verifies threshold signatures on each commitment; alerts if invalid signatures appear.
4. **Consistency Checker**: Samples queries across endpoints and compares results; alerts if data diverges.

---

10. Conclusion

The ACME Protocol's infrastructure architecture demonstrates that Bitcoin-native, fully decentralized systems for critical protocol services are not only feasible but economically practical. By leveraging Bitcoin's consensus, scripting capabilities, and immutable ledger, ACME achieves:

1. **Decentralized API Discovery**: Through blockchain-based naming and verification, eliminating centralized DNS operators while maintaining user-friendly domain-based access.
2. **Trustless Incentive Alignment**: By distributing fees directly via Bitcoin transactions controlled by threshold cryptography, ensuring rewards are earned and claimed without intermediaries or external tokens.
3. **Byzantine Fault Tolerance**: Through threshold signatures requiring consensus above 50%, protecting against malicious minorities while remaining practical for N = 8-16 indexer networks.
4. **Verifiable Correctness**: Through periodic on-chain commitments anchored to Bitcoin's immutable ledger, creating objective proofs that clients can cryptographically verify.

5. **Economic Sustainability**: By batching commitments across epochs, amortizing Bitcoin fees to negligible per-transaction overhead, making continuous on-chain verification economically viable.

The synthesis of these components creates a infrastructure layer that is simultaneously decentralized, trustless, efficient, and aligned with Bitcoin's core security model. Rather than building parallel consensus systems or token economies, ACME piggybacks on Bitcoin's proven security, reducing complexity and external dependencies to their minimum.

Future work might explore:

- Adaptive threshold selection based on Byzantine node detection
- Cross-protocol indexer sharing (multiple protocols sharing a single indexer set)
- Privacy enhancements for fee aggregation
- Zero-knowledge proofs for private state commitments
- Taproot-based script optimizations for more complex conditions

This architecture serves as a template for any protocol seeking Bitcoin-native infrastructure without centralization or external dependencies—demonstrating that truly decentralized infrastructure is achievable and economically viable.

References

Primary Sources

[1] Sovereign Identity: A Comprehensive Guide to Ethereum Name Service (ENS). Tory Green, Medium.
[2] Decentralized Domain Name Systems. Outlier Ventures.
[3] Bitcoin Anchoring. Exonum Documentation.
[4] What is Mintlayer and How Does it Work as a Bitcoin Layer 2? Trust Machines.
[5] Deep dive into Merkle proofs and eth_getProof. Chainstack.
[6] Threshold Signatures. Bitcoin Optech.
[7] OP_RETURN - Storing Data on the Blockchain. Learn Me A Bitcoin.
[8] Hash Timelocked Contracts. University of Alberta Paper.
[9] Design of Non-Custodial BTC Staking. Core DAO Documentation.
[10] DNS Load Balancing For Traffic Distribution. Google SRE Book.
[11] What is Anycast? Cloudflare Learning Center.
[12] Sequencer Commitments. Citrea Technical Specs.
[13] MuSig2: Multi-Signature Schnorr. Bitcoin Optech.

Secondary Sources

[14] Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.

[15] Bünz, B., Bootle, J., Boneh, D., et al. (2018). Bulletproofs: Short Proofs for Confidential Transactions and More.
[16] Boneh, D., Boyen, X., & Shacham, H. (2004). Short Group Signatures.
[17] Poon, A., & Dryja, T. (2015). The Bitcoin Lightning Network.

Document Information

- **Total Pages**: 12
- **Word Count**: ~15,000
- **Figures**: 2 (architecture diagram, timeline diagram)
- **Equations**: Mathematical formulations of threshold signatures and Merkle roots
- **Code Examples**: Pseudocode and Bitcoin script examples throughout

This whitepaper is published for technical discussion and review. Implementation details may vary based on specific deployment requirements.